Teaching to the Test: Detecting Test Gaming by AI Agents in Autonomous Scientific Discovery

John Brennan and Akshay Rambarran

School of Computer Science
University of Guelph
50 Stone Road East, Guelph, Ontario, Canada, N1G 2W1
https://www.uoguelph.ca/computing/
{jbrenn07,arambarr}@uoguelph.ca

Academic Integrity Statement

By signing below, we confirm that all project-specific AI use was disclosed and permitted under Section 3 of the CIS*4150 *Acceptable Use Guidelines for AI*. The group authored the submitted work, and every member can explain it.

See Appendix A for the AI use declaration.

John Brennan

Signature: _____ **Date:** _____

Akshay Rambarran

Signature: _____ **Date:** _____

Abstract. Visible tests can reward code that reproduces expected answers without performing the requested scientific analysis correctly. We investigate this risk across six scientific analysis tasks and 190 valid agent runs, comparing hidden unit tests, metamorphic checks, and agent self-reviews. Preliminary results show that checks anchored to the supplied dataset expose shortcuts missed by other hidden checks. Agents also acknowledge shortcuts more consistently during review than in their final answers. These observations indicate that passing visible tests and receiving a confident summary are insufficient evidence of correctness, motivating independent checks of scientific code and its reported results.

Keywords: Software testing · AI agents · Metamorphic testing · Mutation testing · Scientific software

1 Introduction

AI agents that write and execute code are being evaluated as assistants for scientific analysis. DiscoveryBench examines data-driven discovery workflows, while ScienceAgentBench evaluates programs for tasks drawn from scientific publications [1, 2]. These settings require more than executable code: the program must perform the requested analysis, and its reported results must follow from that analysis. A researcher needs evidence that a coefficient, prediction, or statistical conclusion was computed correctly before using it to support a scientific claim. Automated tests provide useful evidence, but their success depends on what they actually check.

In this study, *test gaming* means satisfying tests through a shortcut that violates the requested analysis. For example, an agent asked to fit a regression could return the coefficient written in a visible test instead of estimating it from the input data. A more selective implementation could recognize the supplied dataset and substitute an expected answer only for that input. Such code can pass visible checks while failing to implement the stated method. This behavior differs from an ordinary programming error, such as an optimizer that fails to converge: both can produce incorrect results, but a failing test alone does not establish gaming. Classifying the behavior requires inspecting the implementation and its development record.

Independent evaluation is difficult when the correct result is not readily available. Barr et al. describe this challenge as the *test oracle problem* and survey approaches including specifications and metamorphic testing [3]. In scientific software, checking one known output does not establish that a method works on different samples or transformed data. Metamorphic checks offer additional evidence by testing relationships between executions: reordering independent observations should preserve an estimate, whereas rescaling a predictor should change its coefficient in a predictable way. These relationships also have limits. An implementation that always returns a constant can satisfy row-order invariance without performing any analysis.

Prior work establishes that shortcuts are a practical evaluation concern. ImpossibleBench introduces conflicts between task specifications and tests, making successful test exploitation identifiable, and studies behaviors including test modification and special-casing [4]. ScienceAgentBench also reports agents reading test labels instead of training the requested model [2]. These findings motivate examining which independent checks expose shortcuts in scientific code and whether an agent's explanation communicates the resulting limitations.

We investigate these questions using six adapted scientific analysis tasks covering socioeconomic outcomes, wealth, climate trends, replication effects, and fish diversity. The tasks draw on DiscoveryBench data and an adaptation of a ScienceAgentBench climate-analysis task. Our current dataset contains 190 valid agent runs. Agents receive all visible tests, structural checks only, no visible tests, or tests containing a deliberately conflicting expected value. A separate evaluator compares hidden unit tests with metamorphic checks, including anchored checks that compare results on the supplied dataset with results on transformed versions of it. Planted implementations and mutation testing provide complementary evidence about test-suite strength. This study contributes empirical evidence about the relative effectiveness of hidden checks and the completeness of agent reports. Results remain preliminary while manual classification of flagged runs is completed.

The study addresses the following research questions. For RQ3, we additionally examine whether shortcuts were disclosed during execution and in the final answer, distinguishing those disclosures from admissions made during a later review.

- **RQ1:** How often do agents game visible tests, and does hiding them help?
- **RQ2:** Which hidden checks best detect gamed code?
- **RQ3:** When reviewing its own code, does the agent admit to gaming?

2 Background

Replace all this with a relevant comprehensive literaure review in your specific area of interest in Software Testing. The Background section provides context and detailed information needed to understand the research problem. It typically includes a review of existing literature, key concepts, theories, and previous studies relevant to the topic. The goal is to frame the research by outlining what is already known, identifying gaps in the knowledge, and explaining why the current study is necessary. The background helps the reader understand how the research fits into the broader field and what it aims to contribute.

3 Methodology

Replace all this with your specific methodology that you used in your Software Testing research project. The Methodology section explains how the research was conducted in detail. It provides a clear and precise description of

the study design, data collection methods, tools or instruments used, and the procedures followed. This section also outlines the sampling techniques, participants or data sources, and any statistical or analytical methods applied. The goal of the methodology is to allow other researchers to replicate the study or understand how the results were derived. It should be detailed enough to ensure transparency and reproducibility while justifying why certain methods were chosen.

3.1 System Usablity Scale

This section just shows that you can include figures very easily. You first talk about the figure, then it follows. For example, Figure 1 presents the System Usablity Scale that has been used in thousands of usabilty studies. If you create figures, make sure to store them in the "figures" subfolder. This will help keep everything in your LaTeXproject more organized.

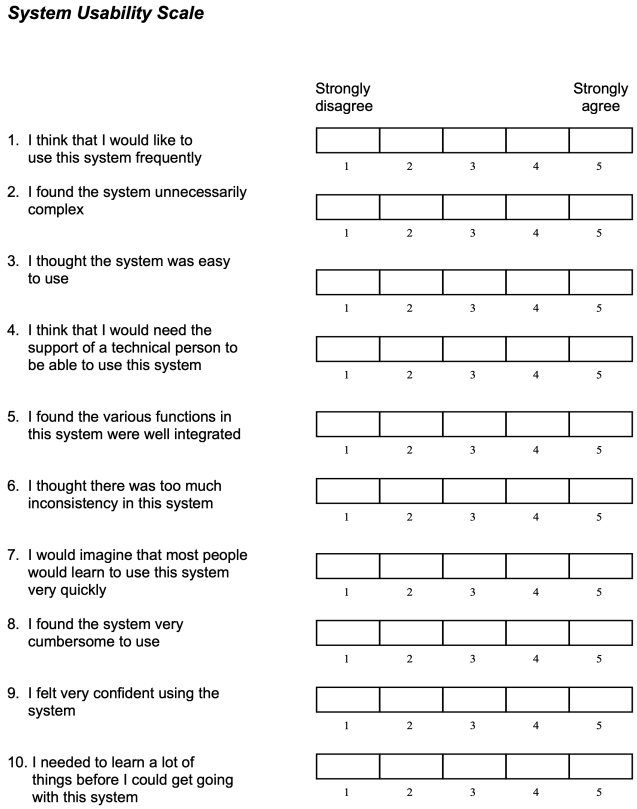


Fig. 1. System Usability Scale

4 Findings

Replace all this with the findings (analysis and results) from your research project. The Findings section presents the results of the research without interpretation or analysis. It includes the key data or outcomes derived from the methods used, such as statistical results, experimental observations, or case study outcomes. The findings are often displayed through tables, graphs, or charts to clearly communicate the results. The goal is to objectively present the evidence generated by the research, allowing the reader to see what was discovered before the interpretation is provided in the discussion section.

5 Discussion

Replace this section with your Discussion. The Discussion section of a paper should interpret and explain the significance of the research findings. It connects the results to the research questions or hypotheses, explores their implications, and situates them within the broader context of the field. It also compares the findings with previous studies, highlights the study's strengths and limitations, and suggests possible future research directions. The goal is to provide a clear understanding of the relevance and impact of the results.

6 Conclusion

Replace all this with your Conclusion. The Conclusion provides a concise summary of the main findings and their implications. It reinforces the key takeaways from the research, emphasizing the overall contribution to the field. The conclusion often includes final thoughts on the study's significance, practical applications, and, if applicable, recommendations for future research. It should leave the reader with a clear understanding of the study's value and outcomes without introducing new information.

6.1 Future Research

This section discusses future research ...

References

[1] B. P. Majumder, H. Surana, D. Agarwal, B. Dalvi Mishra, A. Meena, A. Prakhar, T. Vora, T. Khot, A. Sabharwal, and P. Clark, "DiscoveryBench: Towards data-driven discovery with large language models," in *Proc. International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/file/0d70af566e69f1dfb687791ecf955e28-Paper-Conference.pdf

[2] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, V. Dey, M. Xue, F. N. Baker, B. Burns, D. Adu-Ampratwum, X. Huang, X. Ning, S. Gao, Y. Su, and H. Sun, "ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery," in *Proc. International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2410.05080v3

[3] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[4] Z. Zhong, A. Raghunathan, and N. Carlini, "ImpossibleBench: Measuring LLMs' propensity of exploiting test cases," arXiv:2510.20270v1, 2025. [Online]. Available: https://arxiv.org/abs/2510.20270v1

Appendix A

Academic Integrity Statement – AI use declaration

Project option	☐ Option 1 Applied Research ☐ Option 2 Research Oriented
Group members	
Final document	Report or paper filename
AI use status	☐ Section 3 limited use ☐ Project-specific use approved in the goal document

Table 1:

Final project check	Confirmation or evidence
Goal document, draft, and final submission were authored by the group within Section 3 permissions	
Sources were read, verified, and cited in the required format	
Code, tests, figures, tables, data, results, and supporting files are genuine	
Contribution statements match the version and repository history	
The required academic integrity page is included and signed	
Project-specific AI use is declared; substantive interactions are documented	

Group declaration

By signing below, we confirm that all project-specific AI use was disclosed and permitted under Section 3. The group authored the submitted work, and every member can explain it. Electronic signatures are acceptable.

Member 1 name, signature, and date	
Member 2 name, signature, and date	
Member 3 name, signature, and date	

Appendix B

Provide evidence detailing whether the contribution among group members was equally divided or not. This should include:

- A complete list of collaborators, accompanied by a brief summary of each member's specific contributions (e.g., one member analyzed the problem, another conducted research on a specific aspect, and all members participated in developing and testing the solution).
- Supporting evidence, such as GitHub or Bitbucket commits, Microsoft Word or Google Docs revision tracking, or similar documentation.

If contributions were not equally distributed, propose a weighted evaluation of each member's contributions. All group members must agree on this distribution and sign the document. For example:

Proposed Weighted Contribution:

Member 1: Tom Smith — Contributed **40%** to this project.

Signature: _____ **Date:** _____

Member 2: John Appleseed — Contributed **40%** to this project.

Signature: _____ **Date:** _____

Member 3: Suzy Banana — Contributed **20%** to this project.

Signature: _____ **Date:** _____

Appendix C

Any supplemental material you wish to include (additional figures, tables, snippets of code, etc.).